

TEMA N° 2



DESARROLLO DE APLICACIONES BACKEND



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender la arquitectura y el flujo de datos en el lado del servidor**, reconociendo cómo se procesan las solicitudes web para dar soluciones informáticas reales en nuestro entorno comunitario.
2. **Configurar, administrar y conectar un entorno de servidor local funcional**, integrando lenguajes backend con sistemas de gestión de bases de datos para construir aplicaciones robustas.
3. **Desarrollar un sistema CRUD completo y seguro**, aplicando lógica de programación profesional para resolver necesidades de automatización y gestión de información en el municipio de Pailón.



PRÁCTICA



PREGUNTA DETONANTE

¿Qué sucede realmente en el mundo digital cuando un usuario en el Barrio Central Fátima llena un formulario en su teléfono celular para solicitar un servicio de entrega a domicilio, o cuando la administración del CEA Herman Ortiz Camargo registra tus calificaciones en una plataforma y estas quedan guardadas para siempre? ¿Dónde viven esos datos si el teléfono o la computadora del escritorio se apagan?

PLANTEAMIENTO DEL PROBLEMA TECNOLÓGICO REAL

Imaginemos la siguiente situación en nuestra población de Pailón. El comercio formal en el Barrio Central Fátima está creciendo rápidamente, y las asociaciones de mototaxis del Barrio 24 de Septiembre (cerca de la estación de tren) necesitan coordinar servicios de carga con los productores agrícolas de las zonas colindantes. Actualmente, todo se gestiona anotando datos en cuadernos papel que terminan perdiéndose, mojándose por las lluvias o duplicándose de forma confusa.

Para resolver esto, el CEA Herman Ortiz Camargo, desde su nuevo y moderno módulo educativo en el Barrio Saó, ha decidido encomendar a los estudiantes de cuarto semestre el diseño de una plataforma web local. El objetivo es que una tienda de repuestos del Barrio Central Fátima pueda registrar un pedido de entrega, y que un transportista ubicado en el coliseo municipal del Barrio 27 de Mayo pueda ver esa solicitud en tiempo real en su pantalla.

Como futuro Técnico en Sistemas Informáticos, te encuentras frente a tu computadora en el laboratorio del CEA. Tienes el diseño visual de la página (el Frontend) listo y hermoso, con sus botones y colores bien definidos. Sin embargo, al hacer clic en el botón "Enviar Solicitud", no pasa nada. Los datos se evaporan de la pantalla al recargar la página. Falta el motor oculto que reciba la información, la procese, tome decisiones lógicas y la guarde de forma permanente en un lugar seguro. Ese motor es el Backend, y nuestro desafío es construirlo desde cero para interconectar de forma eficiente los barrios de Pailón.



TEORÍA



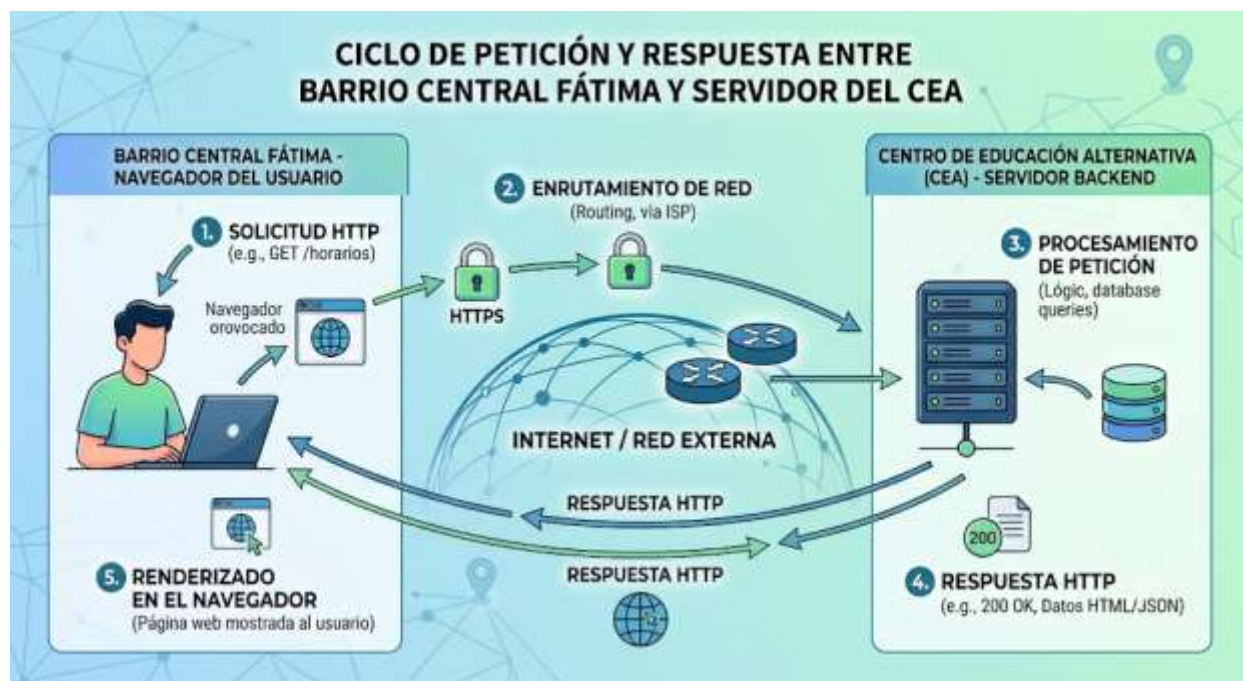
2.1. ARQUITECTURA Y LÓGICA DEL LADO DEL SERVIDOR

Para entender cómo funciona el desarrollo Backend, primero debemos comprender la arquitectura Cliente-Servidor. En el desarrollo web, el **Cliente** (Frontend) es todo aquello con lo que el usuario interactúa directamente: el navegador web (Chrome, Edge, Firefox), las pantallas de los teléfonos celulares y los elementos visuales estructurados con HTML, estilizados con CSS y dinamizados con JavaScript.

Por otro lado, el **Servidor** (Backend) es el sistema operativo, los servicios informáticos, el lenguaje de programación y la base de datos que se ejecutan en una máquina remota o local. Su función principal es procesar las peticiones del cliente, validar la seguridad, ejecutar la lógica del negocio (operaciones matemáticas, validaciones de identidad, procesamiento de archivos) y devolver una respuesta adecuada.

El flujo de comunicación sigue un ciclo estricto de Petición (Request) y Respuesta (Response):

1. **Petición (HTTP Request):** El cliente envía una solicitud al servidor a través del protocolo HTTP/HTTPS. Por ejemplo, un usuario en el Barrio Residencial Norte introduce una URL o presiona un botón para ver los productos disponibles en una tienda pailoneña.
2. **Procesamiento:** El servidor web recibe la solicitud, el lenguaje backend interpreta lo que se está pidiendo, consulta la base de datos si es necesario, y empaqueta el resultado.
3. **Respuesta (HTTP Response):** El servidor envía los datos procesados de vuelta al cliente, generalmente en forma de código HTML estructurado, texto plano o archivos formateados en JSON, para que el navegador del usuario pueda renderizarlos en pantalla.



A diferencia del Frontend, que se ejecuta en el dispositivo del usuario final y depende del rendimiento de su teléfono o computadora, la lógica del Backend se ejecuta completamente en el servidor. Esto significa que como Técnico, tienes el control absoluto del entorno de ejecución, garantizando que las reglas del negocio se cumplan sin importar las limitaciones técnicas del dispositivo del cliente.

2.2. CONFIGURACIÓN DE UN ENTORNO DE SERVIDOR LOCAL (XAMPP/NODE.JS)

Un servidor web real es una computadora encendida las 24 horas del día conectada a internet con una dirección IP pública. Sin embargo, para desarrollar y probar nuestras aplicaciones dentro del laboratorio del CEA Herman Ortiz Camargo sin incurrir en costos de hosting, transformamos nuestras computadoras de escritorio en un **entorno de servidor local**.

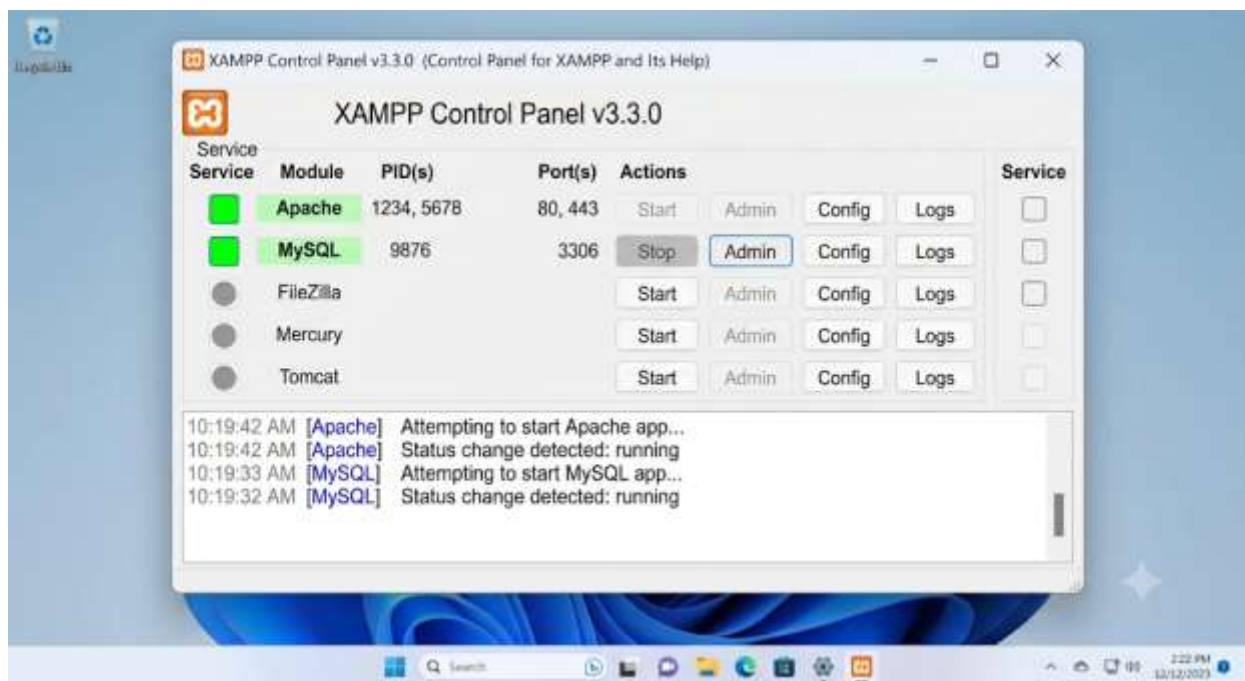
Para este propósito, utilizaremos **XAMPP**, que es un paquete de software libre que agrupa las herramientas esenciales que necesita un Técnico para poner en marcha un backend clásico:

1. **Apache:** El servidor web encargado de procesar las peticiones HTTP.
2. **MySQL/MariaDB:** El sistema de gestión de bases de datos relacionales.
3. **PHP:** El lenguaje de programación del lado del servidor.



Pasos para configurar el entorno local en el CEA:

1. **Instalación:** Ejecutamos el instalador de XAMPP en nuestras computadoras con sistema operativo Windows o Linux, manteniendo las opciones por defecto.
2. **Ubicación de archivos:** Por defecto, XAMPP se instala en la ruta C:\xampp. La carpeta más importante para un desarrollador backend es htdocs (C:\xampp\htdocs). Todo archivo PHP o HTML que coloquemos dentro de esta carpeta será interpretado por el servidor local.
3. **Encendido de servicios:** Abrimos el Panel de Control de XAMPP y hacemos clic en el botón *Start* al lado de Apache y MySQL. Cuando las letras se tornan de color verde, nuestro servidor local está operando.



Para acceder a nuestro servidor desde el navegador web de la computadora, abrimos una pestaña y escribimos la dirección de bucle local: <http://localhost/> o <http://127.0.0.1/>. Si creamos una carpeta llamada *sistema_pailon* dentro de *htdocs*, accederemos a ella escribiendo http://localhost/sistema_pailon/.

2.3. INTRODUCCIÓN A UN LENGUAJE BACKEND (PHP BÁSICO)

PHP (Hypertext Preprocessor) es uno de los lenguajes backend más utilizados a nivel mundial debido a su excelente integración con bases de datos y su curva de aprendizaje amigable para



estudiantes de centros de educación alternativa. Los archivos de PHP se guardan con la extensión .php y pueden mezclar código HTML con lógica de programación utilizando las etiquetas de apertura <?php y de cierre ?>.

A continuación, analizaremos la sintaxis básica mediante un script diseñado para clasificar el estado de un envío de mercancía desde el Barrio Central Fátima hasta el Barrio Saó:

PHP

```
<?php
// Declaración de variables básicas usando el signo de dólar ($)barrioOrigen = "
Barrio Central Fátima";
// Tipo de dato: Cadena de texto (String)barrioDestino = "
Barrio Saó";
// Tipo de dato: Cadena de texto (String)distanciaBloques = 12;
// Tipo de dato: Entero (Integer)esEntregaInmediata = true;
// Tipo de dato: Booleano (Boolean)
// Estructura de control condicional para calcular el tiempo estimadoif
(distanciaBloques <= 5) {
// Si la distancia es menor o igual a 5 bloques, el tiempo es corto
tiempoEstimado = "10 minutos";
}
else if (distanciaBloques > 5 && distanciaBloques <= 15) {
// Si está entre 6 y 15 bloques, el tiempo es moderado    tiempoEstimado = "25
minutos";
}
else {
// Para distancias mayores a las anteriores    tiempoEstimado = "45 minutos";
}
// Imprimir los resultados directamente en el documento HTML devuelto al
clienteecho "<h2>Reporte de Envíos en Pailón</h2>";
echo "<p>El pedido sale desde el <strong>" . barrioOrigen . "</strong> con
destino al <strong>" . barrioDestino . "</strong>.</p>";
echo "<p>Tiempo estimado de llegada del transportista: " . tiempoEstimado .
"</p>";
?>
```

En este bloque de código se observa cómo el backend toma decisiones lógicas basadas en variables de entrada. El usuario final en su navegador no verá las líneas de código PHP ni los condicionales if; únicamente recibirá el texto plano estructurado en HTML generado dinámicamente por la instrucción echo.



2.4. CONEXIÓN DEL BACKEND A UNA BASE DE DATOS LOCAL

Los datos de las variables en PHP son volátiles; cuando el script termina de ejecutarse, la información desaparece de la memoria ram del servidor. Para lograr que los pedidos de los comerciantes de Pailón persistan en el tiempo, debemos conectar nuestro backend con una base de datos relacional MySQL.

Como Técnicos profesionales, utilizaremos la extensión **PDO (PHP Data Objects)**. PDO proporciona una capa de abstracción segura, limpia y orientada a objetos para interactuar con bases de datos, permitiéndonos cambiar de motor de base de datos en el futuro sin reescribir todo nuestro código.

Para establecer la conexión, crearemos un archivo dedicado llamado conexion.php:

PHP

```
<?php
// Configuración de Los parámetros de acceso al servidor de base de
datos servidor = "
localhost";
// Dirección de la máquina donde corre MySQL (nuestra propia PC) usuario = "
root";
// Usuario por defecto configurado por XAMPP password = "";
// Contraseña por defecto en XAMPP (vacía) baseDatos = "
db_pailon";
// Nombre de la base de datos que crearemos en MySQL try {
// Instanciamos un nuevo objeto PDO pasando la cadena de conexión DSN, usuario y
contraseña $conexion = new PDO("
mysql:host=servidor;
dbname=baseDatos;
charset=utf8", usuario, password);
// Configuramos PDO para que lance excepciones en caso de encontrar errores de
sintaxis o conexión $conexion->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
// Mensaje de depuración temporal (comentar o eliminar en producción) // echo
"
Conexión exitosa al sistema de datos de Pailón.";
}
catch (PDOException $error) {
// Si algo falla (ej. base de datos inexistente o datos incorrectos), se captura
el error aquí echo "
Error crítico en la conexión del servidor: " . $error->getMessage();
die();
}
```

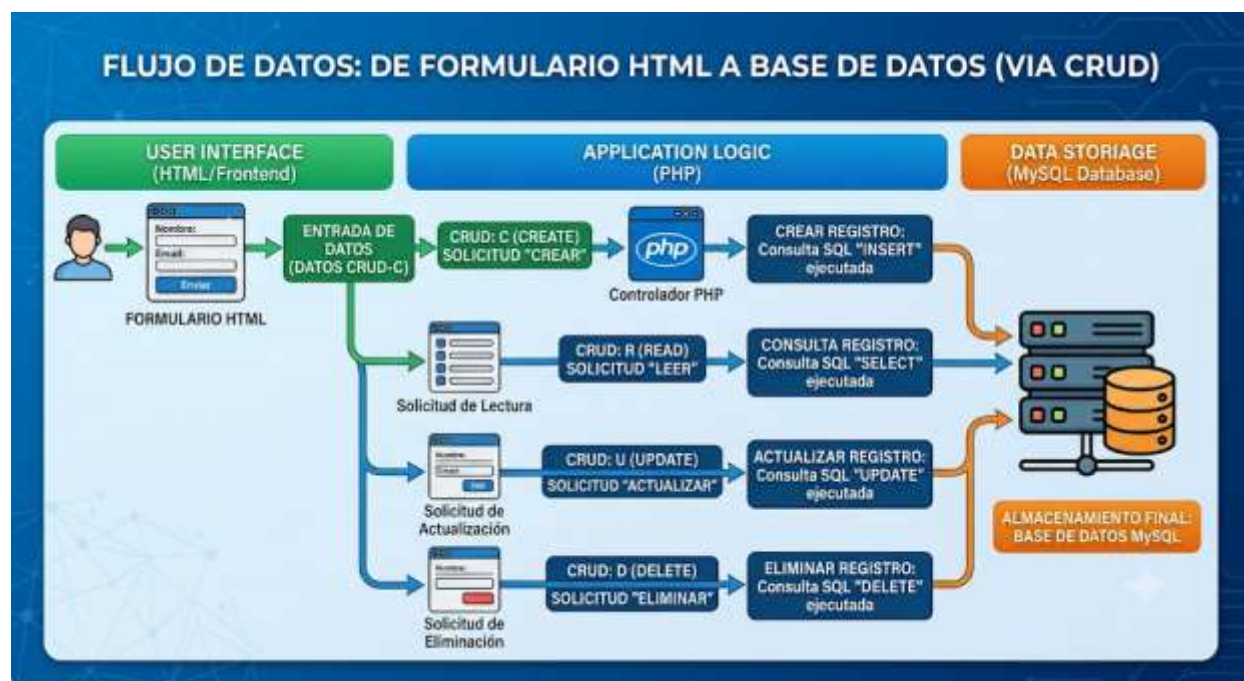


```
// Detiene inmediatamente la ejecución del script backend}
?>
```

Este código implementa un bloque try-catch (intentar-capturar). Si la conexión falla (por ejemplo, si el servicio MySQL de XAMPP está apagado), el servidor no expondrá rutas internas sensibles del sistema, sino que mostrará un mensaje controlado y seguro, protegiendo la infraestructura del CEA.

2.5. EJECUCIÓN DE OPERACIONES CRUD DESDE EL BACKEND

El término **CRUD** es el acrónimo en inglés de las cuatro operaciones fundamentales que un Técnico debe dominar para administrar registros de información en cualquier sistema informático: **C**reate (Crear/Insertar), **R**ead (Leer/Consultar), **U**ppdate (Actualizar/Modificar) y **D**eleate (Borrar/Eliminar).



A continuación, implementaremos la lógica backend completa para gestionar los reportes de reclamos vecinales de Pailón (por ejemplo, baches en el Barrio 27 de Mayo o luminarias rotas en el Barrio Jardines de Pailón) en una tabla de base de datos hipotética llamada reclamos.

Operación CREATE (Insertar nuevos registros)

Este archivo procesa los datos enviados desde un formulario web para guardarlos permanentemente:



PHP

```
<?php
// Incluimos de forma obligatoria el archivo que abre la conexión con
MySQLrequire_once "
conexion.php";
// Simulamos la recepción de datos provenientes de un formulario HTML mediante
el método POSTbarrio = "
Barrio 27 de Mayo";
descripcion = "
Falta de iluminación pública cerca del coliseo municipal";
tecnicoResponsable = "
Juan Choque";
try {
// Preparamos la sentencia SQL utilizando marcadores de posición de parámetros
(?) para evitar hackeos    sql = "
INSERT INTO reclamos (barrio, descripcion, tecnico, fecha_registro) VALUES (?,
?, ?, NOW())";
sentencia = conexion->prepare(sql);
// Ejecutamos la consulta pasando los valores reales organizados en un arreglo
ordenado    sentencia->execute([barrio, descripcion, tecnicoResponsable]);
echo "<h3>Registro Creado Éxito! El reclamo comunitario fue guardado en la base
de datos.</h3>";
}
catch (PDOException e) {
echo "
Error al guardar el reclamo: " . e->getMessage();
}
?>
```

Operación READ (Leer y listar registros existentes)

Este script extrae la información guardada y la organiza en una estructura HTML para que el usuario pueda consultarla desde cualquier pantalla:

PHP

```
<?phprequire_once "
conexion.php";
try {
// Definimos la consulta SQL de selección    sql = "
SELECT id, barrio, descripcion, tecnico, fecha_registro FROM reclamos ORDER BY
id DESC";
sentencia = conexion->prepare(sql);
sentencia->execute();
// Recuperamos todos los registros encontrados y los guardamos en una variable
```



```

asociativa    listaReclamos = sentencia->fetchAll(PDO::ATTR_ASSOC);
echo "<h2>Panel de Control - Reclamos Registrados en Pailón</h2>";
// Recorremos la lista de registros uno a uno utilizando un bucle foreach
foreach (listaReclamos as reclamo) {
echo "<div style='
border: 1px solid #ccc;
padding: 10px;
margin-bottom: 10px;
background-color: #f9f9f9;
'>";
echo "<h4>Código del Reclamo: #" . reclamo['
id'] . " - Ubicación: " . reclamo['
barrio'] . "</h4>";
echo "<p>
<strong>Detalle del problema:</strong> " . reclamo['
descripcion'] . "</p>";
echo "<p>
<small>Asignado al Técnico: " . reclamo['
tecnico'] . " | Registrado el: " . reclamo['
fecha_registro'] . "</small>
</p>";
echo "</div>";
}
}
catch (PDOException e) {
echo "
Error al leer los datos del servidor: " . e->getMessage();
}
?>

```

Operación UPDATE (Actualizar datos de un registro específico)

Permite modificar información guardada previamente introduciendo el identificador único (id) de la fila:

PHP

```

<?phprequire_once "
conexion.php";
// Datos actualizados que el Técnico ingresó en la interfaz de ediciónidReclamo
= 1;
// ID del registro que deseamos modificarnuevoTecnico = "
Mariela Escalante";
// Cambio de técnico asignadonuevoEstado = "
Solucionado";
try {
// Sentencia SQL estructurada para modificar campos específicos filtrando por la

```



```

llave primaria ID    sql = "
UPDATE reclamos SET tecnico = ?, descripcion = CONCAT(descripcion, ' - Estado:
', ?) WHERE id = ?";
sentencia = conexion->prepare(sql);
// Ejecución segura inyectando los parámetros ordenados correspondientes
sentencia->execute([nuevoTecnico, nuevoEstado, idReclamo]);
echo "<p>Registro #" . idReclamo . " actualizado exitosamente en el
backend.</p>";
}
catch (PDOException e) {
echo "
Error al actualizar el registro: " . e->getMessage();
}
?>

```

Operación DELETE (Eliminar de forma definitiva un registro)

Borra un registro de la base de datos de manera física utilizando su clave única. Debe manejarse con extrema precaución:

PHP

```

<?phprequire_once "
conexion.php";
// Identificador del registro que cumple los criterios para ser eliminado del
sistema idEliminar = 1;
try {
// Declaración SQL de borrado estricto con cláusula WHERE obligatoria para
evitar borrar toda la tabla    sql = "
DELETE FROM reclamos WHERE id = ?";
sentencia = conexion->prepare(sql);
// Ejecución del borrado    sentencia->execute([idEliminar]);
echo "<p style='
color: red;
'>El registro número " . idEliminar . " ha sido borrado físicamente del
sistema.</p>";
}
catch (PDOException e) {
echo "
Error al eliminar el registro seleccionado: " . e->getMessage();
}
?>

```



2.6. APIS RESTFUL Y EL INTERCAMBIO DE DATOS EN FORMATO JSON

En la actualidad, las aplicaciones web modernas no siempre recargan páginas HTML completas desde el backend. En su lugar, el servidor funciona como una **API RESTful** (Application Programming Interface), actuando como un intermediario puro de datos.

Una API recibe solicitudes HTTP y, en lugar de renderizar texto o estilos visuales, devuelve únicamente datos crudos estructurados en formato **JSON (JavaScript Object Notation)**. Este formato es ligero, universal y puede ser interpretado tanto por una página web en JavaScript como por una aplicación móvil nativa instalada en un dispositivo Android de un mototaxista del Barrio 24 de Septiembre.

Un archivo PHP configurado para operar como API RESTful utiliza cabeceras HTTP especiales para indicarle al navegador que lo que está viajando por la red no es un documento visual, sino una estructura de datos estructurada:

PHP

```
<?php
// Especificamos al cliente que la respuesta del servidor es de tipo JSON y no
HTMLheader("
Content-Type: application/json;
charset=UTF-8");
// Creamos un arreglo asociativo complejo con información sobre las actividades
en Pailón
datosPailon = [
    "
municipio" => "
Pailón",
    "
provincia" => "
Chiquitos",
    "
cea_principal" => "
Herman Ortiz Camargo",
    "
barrios_activos" => [
        "
Barrio Saó",
        "
Barrio Central Fátima",
        "
Barrio Ovidio Barberi"
    ],
    "
estado_servidor" => "
Operativo",
    "
codigo_respuesta" => 200];
// La función json_encode transforma de manera automática el arreglo PHP en una
cadena de texto JSON válida
echo json_encode(datosPailon);
?>
```



Cuando un programador consume este backend desde el Frontend mediante técnicas asíncronas como fetch en JavaScript, puede procesar la información en milisegundos sin sobrecargar la red local de datos del CEA, logrando sistemas sumamente fluidos y profesionales.

2.7. SEGURIDAD EN EL BACKEND: PREVENCIÓN DE INYECCIONES SQL

Uno de los errores más graves que cometen los desarrolladores principiantes es concatenar variables directamente dentro de cadenas SQL (por ejemplo: "SELECT * FROM usuarios WHERE nombre = '" . usuario . "'""). Si un atacante malintencionado escribe comandos SQL en la caja de texto del formulario web, el servidor los ejecutará de forma ciega, permitiendo el robo de contraseñas, la destrucción de bases de datos o el bypass de inicios de sesión. Esto se conoce como **Inyección SQL (SQLi)**.

Para defendernos como Técnicos calificados, la regla de oro de la seguridad en el backend es: **Nunca confiar en los datos introducidos por el usuario.**

La técnica profesional estándar consiste en el uso de **Sentencias Preparadas (Prepared Statements)** combinadas con marcadores de posición, tal como se mostró en la sección del CRUD. Al separar estrictamente la estructura de la consulta SQL de los datos enviados por el usuario, el motor de la base de datos trata la entrada del cliente únicamente como texto plano, neutralizando por completo cualquier intento de inyección de código malicioso.

2.8. EL ROL DE LA INTELIGENCIA ARTIFICIAL EN LA PROGRAMACIÓN BACKEND CONTEMPORÁNEA

El desarrollo backend está experimentando un cambio drástico con la integración de herramientas de Inteligencia Artificial (IA) generativa como GitHub Copilot, asistentes de lenguaje y entornos de desarrollo inteligentes. Para un Técnico en Sistemas Informáticos egresado del CEA Herman Ortiz Camargo, la IA no sustituye la necesidad de aprender a programar, sino que funciona como un potente copiloto de desarrollo.

En el día a día laboral, un programador backend utiliza modelos de IA para:

1. **Optimización de consultas:** Solicitar esquemas de consultas SQL complejos basados en múltiples tablas relacionales para mejorar los tiempos de respuesta.



2. **Depuración acelerada (Debugging):** Pegar códigos de error arrojados por PDO y recibir explicaciones instantáneas sobre problemas de sintaxis o malas configuraciones de los controladores del servidor.
3. **Generación de pruebas unitarias:** Automatizar la creación de datos de prueba ficticios para verificar si el backend soporta de manera correcta la carga masiva de peticiones Concurrentes de usuarios de diferentes barrios de Pailón.

Comprender la lógica detrás del backend y los flujos de control te capacitará para validar, corregir y liderar proyectos asistidos por IA de manera eficiente y segura.

CIERRE TEÓRICO OBLIGATORIO

❑ *Mitos vs. Realidades*

Mitos sobre el Desarrollo Backend	Realidades del Trabajo del Técnico
"El código backend es el que hace que un sitio web se vea bonito, con colores atractivos y animaciones fluidas."	El backend maneja exclusivamente la lógica oculta, el procesamiento de datos en el servidor y la seguridad. Lo visual corresponde al Frontend.
"Si apago la computadora donde tengo instalado XAMPP, los usuarios de Pailón aún pueden usar el sistema web desde sus casas."	Un servidor local solo funciona mientras la computadora esté encendida y los servicios de XAMPP estén activos en ejecución continua.

❑ *Glosario de Términos*

1. **Backend:** Capa de desarrollo web enfocada en el funcionamiento interno de la aplicación, abarcando el servidor, la lógica del lenguaje y el almacenamiento persistente de datos.
2. **Servidor Web:** Software o hardware diseñado para recibir peticiones HTTP de clientes de red y despachar respuestas procesadas de vuelta.
3. **CRUD:** Acrónimo que clasifica las cuatro operaciones elementales requeridas para administrar el ciclo de vida completo de un registro de datos: Crear, Leer, Actualizar y Borrar.



4. **PDO (PHP Data Objects):** Extensión de PHP ligera y consistente que define una interfaz orientada a objetos para realizar conexiones e interactuar con múltiples motores de bases de datos de forma segura.
5. **JSON:** Formato estándar de intercambio de datos basado en texto plano estructurado en pares de clave-valor, altamente eficiente para la comunicación entre aplicaciones modernas.

□ **Ponte a Prueba**

1. ¿En qué carpeta del entorno local XAMPP debe guardar un Técnico sus archivos de código para que el servidor local pueda interpretarlos de forma correcta?

1. A) C:\xampp\mysql\data
2. B) C:\xampp\htdocs
3. C) C:\Users\Documents
4. *Respuesta Correcta: B*

2. ¿Cuál es el beneficio de utilizar Sentencias Preparadas (Prepared Statements) al interactuar con bases de datos relacionales desde el backend?

1. A) Hacen que el código se ejecute con colores en la interfaz de pantalla del usuario.
2. B) Permiten prescindir de la instalación de XAMPP en el laboratorio del CEA.
3. C) Blindan el sistema contra ataques informáticos de Inyección SQL al separar los datos de la estructura de la consulta.
4. *Respuesta Correcta: C*

3. Si modificas un archivo de extensión .php y agregas código PHP nuevo dentro del laboratorio, ¿qué componente del software local se encarga de procesar las peticiones HTTP y devolver el resultado procesado al navegador?

1. A) El servidor web Apache.
2. B) El motor de base de datos MySQL.



3. C) El editor de texto o IDE de programación.

4. *Respuesta Correcta: A*



VALORACIÓN



El desarrollo de aplicaciones Backend y la administración de servidores otorgan al Técnico en Sistemas Informáticos un gran poder sobre el manejo de la información, lo que conlleva una inmensa responsabilidad ética y social. Cuando creamos bases de datos para un comercio en el Barrio Central Fátima o un registro en el Barrio Ovidio Barberi, estamos manejando datos privados de ciudadanos reales: nombres, teléfonos, direcciones y registros financieros. Un código backend mal estructurado expone esta información confidencial a ciberdelincuentes, dañando la reputación de los emprendedores y violando el derecho a la privacidad de la comunidad pailoneña.

Desde una perspectiva medioambiental, debemos ser conscientes de que el backend tiene una huella ecológica real. Aunque hablemos de la "nube" o el "servidor", el código se ejecuta en máquinas físicas que consumen electricidad las 24 horas del día. Una base de datos mal diseñada, con consultas ineficientes que sobrecargan el procesador del servidor, obliga a los componentes de hardware a consumir más energía y acelera el desgaste térmico de las computadoras, generando residuos electrónicos (*e-waste*) de manera prematura.

Como futuros profesionales formados con los valores del CEA Herman Ortiz Camargo, debemos adoptar la cultura del "Green Computing" (Informática Verde) y el código ético: programar de forma eficiente, optimizar cada consulta SQL para reducir el esfuerzo del procesador, proteger celosamente la integridad de los datos locales de nuestros vecinos y dar un mantenimiento preventivo riguroso a los equipos para alargar su ciclo de vida útil en nuestra población.



PRODUCCIÓN



LABORATORIO PASO A PASO: "SISTEMA DE CONTROL DE PROYECTOS SOCIO-PRODUCTIVOS DEL CEA"

Objetivo: Desarrollar un sistema Backend local operativo que permita crear y listar las propuestas de proyectos técnicos de los estudiantes del CEA Herman Ortiz Camargo, almacenándolos de manera segura en MySQL.

Paso 1: Crear la estructura en el Servidor Local

1. Asegúrate de tener XAMPP encendido (Apache y MySQL en verde).
2. Dirígete a la ruta C:\xampp\htdocs y crea una nueva carpeta llamada proyectos_cea.
3. Dentro de esa carpeta, crea tres archivos vacíos con los siguientes nombres: conexion.php, insertar.php y listar.php.

Paso 2: Crear la Base de Datos en MySQL

1. Abre tu navegador e ingresa a <http://localhost/phpmyadmin/>.
2. Haz clic en la pestaña **Base de datos**, escribe el nombre db_pailon y selecciona el cotejamiento utf8_general_ci. Presiona **Crear**.
3. Haz clic en la pestaña **SQL** y ejecuta el siguiente comando para estructurar la tabla de datos:

SQL

```
CREATE TABLE proyectos (  id INT AUTO_INCREMENT PRIMARY KEY,  titulo VARCHAR(150) NOT NULL,  barrio_impacto VARCHAR(100) NOT NULL,  estudiante VARCHAR(100) NOT NULL,  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

Paso 3: Programar el archivo de Conexión (conexion.php)

Abre el archivo con tu editor de texto favorito (VS Code o Notepad++) y escribe el siguiente código para enlazar PHP con MySQL usando PDO:



PHP

```
<?php
// Parámetros de configuración del servidor Localdsn = "
mysql:host=localhost;
dbname=db_pailon;
charset=utf8";
username = "
root";
password = "";
try {
// Creación de la instancia PDO pdo = new PDO(dsn, username, password);
pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
}
catch (PDOException e) {
echo "
Fallo en la conexión: " . e->getMessage();
die();
}
?>
```

Paso 4: Programar la Inserción de Datos (insertar.php)

Este script simulará la recepción de proyectos reales ideados para beneficiar a los diferentes barrios de nuestra población:

PHP

```
<?php
// Queremos el archivo de conexiónrequire_once "
conexion.php";
// Datos de prueba correspondientes a proyectos reales del CEA Herman Ortiz
CamargotituloProyecto = "
Sistema de Monitoreo de Agua para Huertos Escolares";
barrioImpacto = "
Barrio Saó";
estudianteAutor = "
Carlos Mendoza";
try {
// Preparamos la consulta SQL segura utilizando marcadores de posición sql =
"
INSERT INTO proyectos (titulo, barrio_impacto, estudiante) VALUES (?, ?, ?)";
stmt = pdo->prepare(sql);
// Ejecutamos la inserción inyectando las variables en un arreglo asociativo
stmt->execute([tituloProyecto, barrioImpacto, estudianteAutor]);
echo "<h2>[Backend] Operación Exitosa</h2>";
```



```

echo "<p>Proyecto de Técnico en Sistemas registrado con éxito para el " .
barrioImpacto . "</p>";
echo "<a href='
listar.php'>Ver proyectos guardados</a>";
}
catch (PDOException e) {
echo "
No se pudieron insertar los datos en el servidor: " . e->getMessage();
}
?>

```

Paso 5: Programar la Consulta de Datos (listar.php)

Este script leerá lo almacenado en MySQL y lo presentará de forma clara en pantalla:

PHP

```

<?phprequire_once "
conexion.php";
try {
// Preparamos la consulta SQL para leer todos los registros ordenados    sql = "
SELECT id, titulo, barrio_impacto, estudiante, fecha_creacion FROM proyectos
ORDER BY id DESC";
stmt = pdo->prepare(sql);
stmt->execute();
// Almacenamos el resultado de las filas devueltas    todosProyectos = stmt-
>fetchAll(PDO::ATTR_ASSOC);
echo "<h1>Lista de Proyectos Técnicos Registrados - CEA Herman Ortiz
Camargo</h1>";
echo "<p>Población de Pailón - Santa Cruz, Bolivia</p>";
echo "<hr>";
if (count(todosProyectos) === 0) {
echo "<p>No hay proyectos almacenados en el servidor local todavía.</p>";
}
else {
// Recorremos y mostramos los datos dinámicamente    foreach (todosProyectos
as proyecto) {
echo "<div style='
background-color: #f1f1f1;
border-left: 5px solid #007BFF;
padding: 15px;
margin-bottom: 15px;
'>";
echo "<h3>" . proyecto['
titulo'] . "</h3>";
echo "<p>
<strong>Zona de Impacto:</strong> " . proyecto['
barrio_impacto'] . " | <strong>Desarrollado por:</strong> " . proyecto['

```



```
estudiante'] . "</p>";
echo "<p>
<small>Fecha de Almacenamiento: " . proyecto['
fecha_creacion'] . "</small>
</p>";
echo "</div>";
}
}
echo "<br>
<a href='
insertar.php'>Simular inserción de otro proyecto</a>";
}
catch (PDOException e) {
echo "
Error de lectura de datos en el backend: " . e->getMessage();
}
?>
```

Verificación: Abre tu navegador e ingresa a http://localhost/proyectos_cea/insertar.php para ingresar datos automáticos, y luego haz clic en el enlace para visualizar cómo los datos viajan y persisten a través de la arquitectura backend que has construido.

RECURSOS ADICIONALES

1. **Manual Oficial de PHP y PDO:** Documentación oficial y completa para comprender a fondo la sintaxis, métodos avanzados de manejo de objetos de datos y configuraciones de seguridad del lenguaje. Visitar: <https://www.php.net/manual/es/book.pdo.php>
2. **MDN Web Docs - Lado del Servidor / Backend:** Guía introductoria de Mozilla ideal para estudiantes, donde se explican detalladamente los conceptos globales de arquitectura, protocolos HTTP, cabeceras y manejo seguro de datos en aplicaciones web. Visitar: <https://developer.mozilla.org/es/docs/Learn/Server-side>
3. **XAMPP Open Source Project:** Sitio oficial para realizar descargas limpias del entorno de desarrollo local, acceso a foros de ayuda de la comunidad global de desarrolladores y guías de configuración avanzada para sistemas Apache y MySQL. Visitar: <https://www.apachefriends.org/es/index.html>